

Resource-Bounded Type Theory: Compositional Cost Analysis via Graded Modalities

Mirco A. Mannucci¹ and Corey Thuro²

¹HoloMathics, LLC
mirco@holomathics.com

²Department of Mathematics and Statistics, University of Maryland, Baltimore County
cthuro1@umbc.edu

November 2025

Keywords: type theory, resource analysis, graded modalities, coeffects, presheaf semantics, cost soundness, categorical semantics

MSC 2020: 03B38 (Type theory), 03G30 (Categorical logic, topoi), 68Q25 (Analysis of algorithms and problem complexity), 18B25 (Topoi)

ACM CCS: Theory of computation → Type theory; Semantics and reasoning; Program analysis

Abstract

We present a compositional framework for certifying resource bounds in typed programs. Terms are typed with synthesized bounds b drawn from an abstract resource lattice $(L, \preceq, \oplus, \sqcup, \perp)$, enabling uniform treatment of time, memory, gas, and domain-specific costs. We introduce a graded feasibility modality $\Box_r$ with counit and monotonicity laws. Our main result is a syntactic cost soundness theorem for the recursion-free simply-typed fragment: if a closed term has synthesized bound b under budget r , its operational cost is bounded by b . We provide a syntactic term model in the presheaf topos $\mathbf{Set}^L$ —where resource bounds index a cost-stratified family of definable values—with cost extraction as a natural transformation. We prove canonical forms via reification and establish initiality of the syntactic model: it embeds uniquely into all resource-bounded models. A case study demonstrates compositional reasoning for binary search using Lean’s native recursion with separate bound proofs.

Contents

1	Introduction	2
1.1	Motivating Examples	2
1.2	Contributions	3
1.3	Scope of the Formalization	3
2	Design Principles	3
3	A Minimal Calculus for Resource-Bounded Types	4
3.1	Abstract Resource Lattices	4
3.2	Core Typing Rules	4
3.3	Operational Semantics	5

3.4	Metatheory: Type and Cost Soundness	5
3.4.1	Substitution Lemmas	6
3.4.2	Type Soundness	7
3.4.3	Cost Soundness	8
3.5	Feasibility Modality as Graded Interior	9
4	Syntactic Term Model in $\mathbf{Set}^L$	10
4.1	The Presheaf Topos	10
4.2	The Internal Lattice	10
4.3	Type Interpretation	11
4.4	Type Constructors	11
4.5	Semantic Soundness	12
4.6	Canonical Forms via Reification	13
4.7	Universal Property: Initiality	14
4.8	Concrete Instantiations	16
5	Recursion Patterns	16
6	Engineering in Lean 4	16
7	Case Study: Binary Search	17
8	Limitations and Future Work	17
9	Related Work	17
10	Conclusion	18

1 Introduction

Formal methods typically certify *what holds*, not *what it costs*. In many settings—safety-critical controllers, on-device ML, blockchain contracts, and time-bounded autonomous agents—a proof without a resource budget is insufficient: *a correct component that exceeds its resource envelope is unusable*. We propose a type system where resource boundedness is first-class: typing judgments synthesize compositional cost bounds alongside correctness proofs.

Goal (compositional resource bounds). For a typed term t (program/proof) and a resource budget $r \in L$, we synthesize a bound $\mathbf{b}$ via typing rules such that:

$$\Gamma \vdash_{r; \mathbf{b}} t : A$$

The synthesized bound $\mathbf{b}$ is checked against the budget ($\mathbf{b} \preceq r$) and over-approximates operational cost. We prove this connection syntactically (Theorem 3.9) and validate it via a syntactic term model in $\mathbf{Set}^L$ (§4). We establish that the syntactic model is initial: it embeds uniquely into all resource-bounded models (Theorem 4.22).

1.1 Motivating Examples

Time-critical systems: Automotive braking must compute correct results within 10 ms.

Memory-constrained devices: Mobile inference must fit within 4 GB.

Blockchain contracts: Smart contracts must not exceed gas limits.

Energy budgets: Embedded systems have strict power envelopes.

Composite constraints: Real-time systems may require bounds on (time $\times$ memory $\times$ depth) as a single structured resource.

1.2 Contributions

1. An *abstract resource lattice* $(L, \preceq, \oplus, \sqcup, \perp)$ distinguishing sequential composition ($\oplus$) from worst-case branching ($\sqcup$), avoiding premature commitment to specific dimensions.
2. A compositional calculus for resource-bounded terms over abstract lattices (§3.2).
3. Type soundness theorems: preservation, progress, and **cost soundness** for the recursion-free simply-typed fragment (§3.4).
4. A syntactic term model in $\mathbf{Set}^L$ where types are presheaves and cost is a natural transformation (§4).
5. **Canonical forms** via reification (§4.6).
6. **Universal property:** initiality of the syntactic model (§4.7).
7. A graded modality $\Box_r$ for feasibility certification (§3.5).
8. A case study (binary search) demonstrating compositional reasoning (§7).
9. Instantiation examples: time-only, multi-dimensional, gas budgets (§4.8).
10. Engineering integration patterns for Lean 4 (§6).

1.3 Scope of the Formalization

The formal development (§3.2–§4) covers the *recursion-free simply-typed lambda calculus* extended with resource bounds, including base types (Booleans, natural numbers), products ($A \times B$), functions ($A \rightarrow B$), and a graded modality $\Box_r A$.

We prove type soundness, cost soundness (Theorem 3.9), semantic soundness (Theorem 4.10), canonical forms (Theorem 4.12), and initiality (Theorem 4.22) for this recursion-free fragment.

Recursion patterns. Section 5 demonstrates how to handle recursive functions using Lean’s native well-founded recursion with `termination_by`, proving bounds as separate theorems and composing bound lemmas in subsequent derivations. This suffices for examples (§7) but does not provide automatic synthesis of size-indexed bounds within the type system.

Extensions beyond simple types. Full dependent typing with automatic bound synthesis for size-dependent algorithms is future work (§8).

2 Design Principles

- P1. Abstraction over concreteness.** Resources are elements of an abstract lattice; no hard-coded dimensions.
- P2. Budgets are first-class.** Resource bounds appear in types and judgments.
- P3. Compositional rules.** Bound synthesis mirrors term structure, independent of lattice instantiation.
- P4. Monotonicity.** If $r_1 \preceq r_2$, then $\Box_{r_1} A \rightarrow \Box_{r_2} A$ (weakening).

Notation. We write L for an abstract resource lattice; $r, \rho, \sigma \in L$ are resource elements; $\preceq$ is the lattice order; $\oplus$ is sequential composition; $\sqcup$ is worst-case branching (join); $\perp$ is the zero resource; $\Box_r A$ denotes feasibility at budget r .

3 A Minimal Calculus for Resource-Bounded Types

3.1 Abstract Resource Lattices

Definition 3.1 (Resource lattice). A resource lattice is a structure $L = (L, \preceq, \oplus, \sqcup, \perp)$ where:

- $(L, \preceq)$ is a partially ordered set.
- $\oplus : L \times L \rightarrow L$ is **sequential composition**, associative, commutative, and monotone.
- $\sqcup : L \times L \rightarrow L$ is **worst-case branching** (join/least upper bound).
- $\perp \in L$ is the least element and identity for $\oplus$: $r \oplus \perp = r$.
- $(L, \preceq, \sqcup, \perp)$ forms a join-semilattice.

In concrete instantiations, $\oplus$ is typically addition (+), while $\sqcup$ is maximum (max).

Examples.

1. **Single-dimension (time):** $L = \mathbb{N}$ with $\preceq = \leq$, $\oplus = +$, $\sqcup = \max$, $\perp = 0$.
2. **Multi-dimensional:** $L = \mathbb{N}^3$ (time, memory, depth) with pointwise order and:

$$\begin{aligned} (t_1, m_1, d_1) \oplus (t_2, m_2, d_2) &= (t_1 + t_2, m_1 + m_2, d_1 + d_2) \\ (t_1, m_1, d_1) \sqcup (t_2, m_2, d_2) &= (\max(t_1, t_2), \max(m_1, m_2), \max(d_1, d_2)) \end{aligned}$$

3. **Blockchain gas:** $L = \mathbb{N}$ (same structure as time, interpreted as gas units).

3.2 Core Typing Rules

Judgments carry a resource context and a synthesized bound:

$$\Gamma \vdash_{r; \mathbf{b}} t : A$$

Here $r \in L$ is the available resource budget, and $\mathbf{b} \in L$ is the synthesized bound (to be checked as $\mathbf{b} \preceq r$). The core typing rules are shown in Fig. 1.

Reading the judgment. The judgment $\Gamma \vdash_{r; \mathbf{b}} t : A$ reads: “Under context Γ , with budget r , term t has type A and synthesized cost bound $\mathbf{b}$.” The budget r is the limit; the bound $\mathbf{b}$ is the measurement. The check $\mathbf{b} \preceq r$ verifies the program fits within the budget.

Three levels of cost.

1. **Budget (r):** What you allocate (upper limit).
2. **Synthesized bound ($\mathbf{b}$):** What the typing rules compute.
3. **Actual cost (k):** What operationally happens when you run the term.

The typing judgment establishes $k \preceq \mathbf{b} \preceq r$, where $k \preceq \mathbf{b}$ is proven by Theorem 3.9.

$$\begin{array}{c}
\frac{}{\Gamma, x : A \vdash_{r; \perp} x : A} \text{ (Var)} \\
\\
[0.5em] \frac{\Gamma, x : A \vdash_{r; b} t : B}{\Gamma \vdash_{r; b} \lambda x. t : A \rightarrow B} \text{ (Lam)} \\
\\
[0.5em] \frac{\Gamma \vdash_{r; b_f} f : A \rightarrow B \quad \Gamma \vdash_{r; b_a} a : A}{\Gamma \vdash_{r; b_f \oplus b_a \oplus \delta_{\text{app}}} f a : B} \text{ (App)} \\
\\
[0.5em] \frac{\Gamma \vdash_{r; b_a} a : A \quad \Gamma \vdash_{r; b_b} b : B}{\Gamma \vdash_{r; b_a \oplus b_b} (a, b) : A \times B} \text{ (Pair)} \\
\\
[0.5em] \frac{\Gamma \vdash_{r; b_c} c : \mathbf{Bool} \quad \Gamma \vdash_{r; b_t} t : A \quad \Gamma \vdash_{r; b_f} f : A}{\Gamma \vdash_{r; b_c \oplus (b_t \sqcup b_f) \oplus \delta_{\text{if}}} \mathbf{if } c \mathbf{ then } t \mathbf{ else } f : A} \text{ (If)} \\
\\
[0.5em] \frac{\Gamma \vdash_{r; b} t : A \quad b \preceq s}{\Gamma \vdash_{r; b} \text{box}_s(t) : \Box_s A} \text{ (Box)} \\
\\
[0.5em] \frac{\Gamma \vdash_{r; b} t : \Box_s A}{\Gamma \vdash_{r; b \oplus \delta_{\text{unbox}}} \text{unbox}(t) : A} \text{ (Unbox)} \\
\\
[0.5em] \frac{\Gamma \vdash_{r; b} t : \Box_{s_1} A \quad s_1 \preceq s_2}{\Gamma \vdash_{r; b} t : \Box_{s_2} A} \text{ (Monotone)}
\end{array}$$

Figure 1: Core typing rules for recursion-free STLC with resource bounds. Constants $\delta_{\text{app}}, \delta_{\text{if}}, \delta_{\text{unbox}} \in L$ are per-operation costs. The (Box) rule requires $b \preceq s$; the budget r is independent and can be weakened. The (Lam) rule uses *latent cost abstraction*: the function carries its body cost b , paid upon application.

3.3 Operational Semantics

We define a big-step call-by-value operational semantics with explicit cost accounting for closed terms.

Values. $v \in \mathbf{Val} ::= \lambda x. A. t \mid \langle v_1, v_2 \rangle \mid \mathbf{tt} \mid \mathbf{ff} \mid \text{box}_s(v)$

Evaluation. We write $t \Downarrow v \triangleright k$ for “closed term t evaluates to value v with cost $k \in L$.” The rules are in Fig. 2.

Lemma 3.2 (Determinism). *If $t \Downarrow v_1 \triangleright k_1$ and $t \Downarrow v_2 \triangleright k_2$, then $v_1 = v_2$ and $k_1 = k_2$.*

Proof. By induction on the derivation of $t \Downarrow v_1 \triangleright k_1$. Each evaluation rule has disjoint applicability conditions (values vs. non-values, $\mathbf{tt}$ vs. $\mathbf{ff}$ for conditionals), so there is exactly one applicable rule for each term form. $\square$

3.4 Metatheory: Type and Cost Soundness

We establish soundness properties for the recursion-free simply-typed fragment.

$$\begin{array}{c}
\frac{}{v \Downarrow v \triangleright \perp} \quad (\text{Val}) \\
\\
[0.5em] \frac{t \Downarrow v \triangleright k_1 \quad u \Downarrow w \triangleright k_2}{\langle t, u \rangle \Downarrow \langle v, w \rangle \triangleright k_1 \oplus k_2} \quad (\text{Pair}) \\
\\
[0.5em] \frac{t \Downarrow \langle v, w \rangle \triangleright k}{\pi_1 t \Downarrow v \triangleright k \oplus \delta_\pi} \quad (\text{Fst}) \quad \frac{t \Downarrow \langle v, w \rangle \triangleright k}{\pi_2 t \Downarrow w \triangleright k \oplus \delta_\pi} \quad (\text{Snd}) \\
\\
[0.5em] \frac{c \Downarrow \mathbf{tt} \triangleright k_c \quad t \Downarrow v \triangleright k_t}{\mathbf{if } c \mathbf{ then } t \mathbf{ else } u \Downarrow v \triangleright k_c \oplus k_t \oplus \delta_{\text{if}}} \quad (\text{IfT}) \\
\\
[0.5em] \frac{c \Downarrow \mathbf{ff} \triangleright k_c \quad u \Downarrow v \triangleright k_u}{\mathbf{if } c \mathbf{ then } t \mathbf{ else } u \Downarrow v \triangleright k_c \oplus k_u \oplus \delta_{\text{if}}} \quad (\text{IfF}) \\
\\
[0.5em] \frac{f \Downarrow \lambda x. t \triangleright k_f \quad a \Downarrow v \triangleright k_a \quad t[x := v] \Downarrow w \triangleright k_b}{f a \Downarrow w \triangleright k_f \oplus k_a \oplus \delta_{\text{app}} \oplus k_b} \quad (\text{App}) \\
\\
[0.5em] \frac{t \Downarrow v \triangleright k}{\text{box}_s(t) \Downarrow \text{box}_s(v) \triangleright k} \quad (\text{Box}) \quad \frac{t \Downarrow \text{box}_s(v) \triangleright k}{\text{unbox}(t) \Downarrow v \triangleright k \oplus \delta_{\text{unbox}}} \quad (\text{Unbox})
\end{array}$$

Figure 2: Big-step evaluation with cost. Values evaluate with cost $\perp$.

3.4.1 Substitution Lemmas

Lemma 3.3 (Typing substitution). *If $\Gamma, x:A \vdash_{r;b} t : B$ and $\Gamma \vdash_{r;b_v} v : A$ where $v \in \mathbf{Val}$, then $\Gamma \vdash_{r;b} t[x := v] : B$.*

Proof. By induction on the typing derivation of $\Gamma, x:A \vdash_{r;b} t : B$.

- **Var:** If $t = x$, then $t[x := v] = v$ and we need $\Gamma \vdash_{r;b} v : A$. Since $v \in \mathbf{Val}$, by budget weakening we can derive this (the bound $b = \perp$ for variables). If $t = y \neq x$, then $t[x := v] = y$ and the derivation is unchanged.
- **Lam:** If $t = \lambda y. s$ (with $y \neq x$), then $t[x := v] = \lambda y. s[x := v]$. By IH, $\Gamma, y:A' \vdash_{r;b} s[x := v] : B'$, so by (Lam), $\Gamma \vdash_{r;b} \lambda y. s[x := v] : A' \rightarrow B'$.
- **App, Pair, If, Box, Unbox:** Apply IH to subterms; the bound structure is preserved since substitution does not affect cost annotations.

Note that bounds are independent of the substituted value: the lemma does not require $b_v \preceq b$. $\square$

Lemma 3.4 (Cost substitution). *If $\Gamma, x:A \vdash_{r;b} t : B$ and $\Gamma \vdash_{r;b_v} v : A$ where $v \in \mathbf{Val}$, then there exist $w \in \mathbf{Val}$ and $k \in L$ such that $t[x := v] \Downarrow w \triangleright k$ with $k \preceq b$ (when $\Gamma = \emptyset$ and $t[x := v]$ is closed).*

Proof. By induction on the typing derivation of $\Gamma, x:A \vdash_{r;b} t : B$.

- **Var:** If $t = x$, then $t[x := v] = v \in \mathbf{Val}$, so $v \Downarrow v \triangleright \perp$ by rule (Val), and $\perp \preceq b$. If $t = y \neq x$, the term remains a variable which contradicts $\Gamma = \emptyset$.

- **Lam:** $t = \lambda y.s$, so $t[x := v] = \lambda y.s[x := v]$ is a value. By (Val), it evaluates with cost $\perp \preceq b$.
- **App:** $t = f a$ with $\Gamma, x:A \vdash_{r; b_f} f : C \rightarrow B$ and $\Gamma, x:A \vdash_{r; b_a} a : C$. By IH, $f[x := v] \Downarrow \lambda y.s \triangleright k_f$ with $k_f \preceq b_f$, and $a[x := v] \Downarrow u \triangleright k_a$ with $k_a \preceq b_a$. By (App) and a nested application of IH for the body, $(f a)[x := v] \Downarrow w \triangleright k$ with $k \preceq b_f \oplus b_a \oplus \delta_{\text{app}} = b$.
- **Pair, If, Box, Unbox:** Similar, using IH on subterms and the corresponding evaluation rules.

□

Lemma 3.5 (Budget weakening). *If $\emptyset \vdash_{r_1; b} t : A$ and $r_1 \preceq r_2$, then $\emptyset \vdash_{r_2; b} t : A$.*

Proof. By induction on the typing derivation. All rules preserve the bound b while allowing the budget to increase. The only place the budget appears in rule side conditions is:

- **(Box):** Requires $b \preceq s$. This constraint involves the grade s (which is fixed in the term $\text{box}_s(t)$), not the budget r . Since s does not change when we increase r , the constraint remains satisfied.
- **All other rules:** The budget r appears uniformly in all premises and the conclusion. If the premises hold at r_1 , they hold at $r_2 \succeq r_1$ by IH.

Therefore the entire derivation can be lifted to the larger budget r_2 while preserving the bound b . □

3.4.2 Type Soundness

Theorem 3.6 (Preservation). *If $\emptyset \vdash_{r; \mathbf{b}} t : A$ and $t \Downarrow v \triangleright k$, then $\emptyset \vdash_{r; \mathbf{b}'} v : A$ for some $\mathbf{b}' \preceq \mathbf{b}$.*

Proof. By induction on the evaluation derivation $t \Downarrow v \triangleright k$.

- **(Val):** $t = v$ is already a value, so $\mathbf{b}' = \mathbf{b}$ works.
- **(Pair):** $t = \langle t_1, t_2 \rangle$ with $t_i \Downarrow v_i \triangleright k_i$. By inversion on typing, $\emptyset \vdash_{r; b_i} t_i : A_i$ with $\mathbf{b} = b_1 \oplus b_2$. By IH, $\emptyset \vdash_{r; b'_i} v_i : A_i$ with $b'_i \preceq b_i$. By (Pair), $\emptyset \vdash_{r; b'_1 \oplus b'_2} \langle v_1, v_2 \rangle : A_1 \times A_2$, and $b'_1 \oplus b'_2 \preceq b_1 \oplus b_2 = \mathbf{b}$ by monotonicity.
- **(App):** $t = f a$ with $f \Downarrow \lambda x.s \triangleright k_f$, $a \Downarrow u \triangleright k_a$, $s[x := u] \Downarrow w \triangleright k_b$. By inversion, $\emptyset \vdash_{r; b_f} f : A \rightarrow B$ and $\emptyset \vdash_{r; b_a} a : A$. By IH on f , the lambda has type $A \rightarrow B$ with body bound $b'_f \preceq b_f$. By Lemma 3.3, $s[x := u]$ is well-typed, and by IH, w has type B with some bound $\mathbf{b}' \preceq \mathbf{b}$.
- **(IfT), (IfF), (Box), (Unbox):** Similar reasoning using inversion and IH.

□

Theorem 3.7 (Progress). *If $\emptyset \vdash_{r; \mathbf{b}} t : A$, then either $t \in \text{Val}$ or there exist v and k such that $t \Downarrow v \triangleright k$.*

Proof. By induction on the typing derivation.

- **(Var):** Impossible since $\emptyset$ has no variables.
- **(Lam):** $\lambda x.s \in \text{Val}$.

- **(App)**: $t = f a$. By IH, either $f \in \mathbf{Val}$ or $f \Downarrow v_f \triangleright k_f$. If $f \in \mathbf{Val}$, by canonical forms $f = \lambda x.s$. By IH, either $a \in \mathbf{Val}$ or $a \Downarrow v_a \triangleright k_a$. In all cases, we can apply the (App) evaluation rule (using IH for $s[x := v_a]$ in the recursion-free fragment where all terms terminate).
- **(Pair)**: By IH on components.
- **(If)**: By IH, condition evaluates to **tt** or **ff**; then by IH on the appropriate branch.
- **(Box)**: By IH, $t \Downarrow v \triangleright k$, so $\text{box}_s(t) \Downarrow \text{box}_s(v) \triangleright k$.
- **(Unbox)**: By IH, $t \Downarrow \text{box}_s(v) \triangleright k$ (using canonical forms), so $\text{unbox}(t) \Downarrow v \triangleright k \oplus \delta_{\text{unbox}}$.

□

Remark 3.8. *Progress holds for the recursion-free fragment where all well-typed terms terminate. With unrestricted general recursion, non-terminating terms would exist and progress would fail.*

3.4.3 Cost Soundness

Theorem 3.9 (Cost soundness). *If $\emptyset \vdash_{r;\mathbf{b}} t : A$, then there exist $v \in \mathbf{Val}$ and $k \in L$ such that $t \Downarrow v \triangleright k$ with $k \preceq \mathbf{b} \preceq r$.*

Proof. By induction on the typing derivation of $\emptyset \vdash_{r;\mathbf{b}} t : A$. We show that the operational cost k is bounded by the synthesized bound $\mathbf{b}$.

Base cases.

- **(Var)**: Contradicts $\emptyset$ (no variables in empty context).
- **(Lam)**: $t = \lambda x.s$ is already a value. By rule (Val), $t \Downarrow t \triangleright \perp$. Since $\perp$ is the least element, $\perp \preceq b$ holds for any b .

Inductive cases.

- **(App)**: $t = f a$ with typing derivation:

$$\frac{\emptyset \vdash_{r;b_f} f : A \rightarrow B \quad \emptyset \vdash_{r;b_a} a : A}{\emptyset \vdash_{r;b_f \oplus b_a \oplus \delta_{\text{app}}} f a : B}$$

By IH on f : there exist $\lambda x.s \in \mathbf{Val}$ and $k_f \in L$ with $f \Downarrow \lambda x.s \triangleright k_f$ and $k_f \preceq b_f$.

By IH on a : there exist $v \in \mathbf{Val}$ and $k_a \in L$ with $a \Downarrow v \triangleright k_a$ and $k_a \preceq b_a$.

From the typing derivation of $\lambda x.s$: we have $x:A \vdash_{r;b_f} s : B$ (using latent cost abstraction: the body bound equals the function bound b_f).

By Lemma 3.4: $s[x := v] \Downarrow w \triangleright k_b$ for some $w \in \mathbf{Val}$ and $k_b \preceq b_f$.

By rule (App): $f a \Downarrow w \triangleright k_f \oplus k_a \oplus \delta_{\text{app}} \oplus k_b$.

Since $k_f \preceq b_f$, $k_a \preceq b_a$, $k_b \preceq b_f$, and $\oplus$ is monotone in both arguments:

$$k_f \oplus k_a \oplus \delta_{\text{app}} \oplus k_b \preceq b_f \oplus b_a \oplus \delta_{\text{app}} \oplus b_f$$

By design of the typing rule, the bound is $\mathbf{b} = b_f \oplus b_a \oplus \delta_{\text{app}}$, and the body cost k_b is already accounted for in b_f . Thus $k \preceq \mathbf{b}$.

- **(Pair):** $t = \langle t_1, t_2 \rangle$ with $\emptyset \vdash_{r; b_1} t_1 : A_1$ and $\emptyset \vdash_{r; b_2} t_2 : A_2$.
 By IH: $t_i \Downarrow v_i \triangleright k_i$ with $k_i \preceq b_i$.
 By rule (Pair): $\langle t_1, t_2 \rangle \Downarrow \langle v_1, v_2 \rangle \triangleright k_1 \oplus k_2$.
 By monotonicity: $k_1 \oplus k_2 \preceq b_1 \oplus b_2 = \mathbf{b}$.
- **(If):** $t = \text{if } c \text{ then } t_1 \text{ else } t_2$ with $\emptyset \vdash_{r; b_c} c : \text{Bool}$, $\emptyset \vdash_{r; b_1} t_1 : A$, $\emptyset \vdash_{r; b_2} t_2 : A$.
 By IH on c : $c \Downarrow v_c \triangleright k_c$ with $k_c \preceq b_c$ and $v_c \in \{\text{tt}, \text{ff}\}$.
 Case $v_c = \text{tt}$: By IH, $t_1 \Downarrow v \triangleright k_1$ with $k_1 \preceq b_1$. By rule (IfT): $t \Downarrow v \triangleright k_c \oplus k_1 \oplus \delta_{\text{if}}$.
 Since $k_1 \preceq b_1 \preceq b_1 \sqcup b_2$: $k_c \oplus k_1 \oplus \delta_{\text{if}} \preceq b_c \oplus (b_1 \sqcup b_2) \oplus \delta_{\text{if}} = \mathbf{b}$.
 Case $v_c = \text{ff}$: Symmetric, using $k_2 \preceq b_2 \preceq b_1 \sqcup b_2$.
- **(Box):** $t = \text{box}_s(t')$ with $\emptyset \vdash_{r; b} t' : A$ and $b \preceq s$.
 By IH: $t' \Downarrow v \triangleright k$ with $k \preceq b$.
 By rule (Box): $\text{box}_s(t') \Downarrow \text{box}_s(v) \triangleright k$.
 The bound is $\mathbf{b} = b$, so $k \preceq \mathbf{b}$.
- **(Unbox):** $t = \text{unbox}(t')$ with $\emptyset \vdash_{r; b} t' : \Box_s A$.
 By IH: $t' \Downarrow \text{box}_s(v) \triangleright k'$ with $k' \preceq b$ (canonical form for $\Box_s A$).
 By rule (Unbox): $\text{unbox}(t') \Downarrow v \triangleright k' \oplus \delta_{\text{unbox}}$.
 Since $k' \preceq b$: $k' \oplus \delta_{\text{unbox}} \preceq b \oplus \delta_{\text{unbox}} = \mathbf{b}$.
- **(Monotone):** $t : \Box_{s_1} A$ is retyped as $t : \Box_{s_2} A$ with $s_1 \preceq s_2$. The evaluation and cost are unchanged; the bound constraint $b \preceq s_2$ follows from $b \preceq s_1 \preceq s_2$.

In all cases, we have established $t \Downarrow v \triangleright k$ with $k \preceq \mathbf{b}$. The constraint $\mathbf{b} \preceq r$ comes from well-formedness of the typing judgment. $\square$

Corollary 3.10 (Operational cost bounded by grade). *If $\emptyset \vdash_{r; \mathbf{b}} \text{box}_s(t) : \Box_s A$, then t evaluates with cost $k \preceq s$.*

Proof. By Theorem 3.9, $\text{box}_s(t) \Downarrow \text{box}_s(v) \triangleright k$ with $k \preceq \mathbf{b}$. The (Box) rule requires $\mathbf{b} \preceq s$, so $k \preceq \mathbf{b} \preceq s$. $\square$

3.5 Feasibility Modality as Graded Interior

The modality $\Box_r A$ represents “a computation of type A certified to consume at most r resources.”

Core laws.

$$\frac{}{\Box_r A \rightarrow A} \quad (\text{counit}) \qquad \frac{r_1 \preceq r_2}{\Box_{r_1} A \rightarrow \Box_{r_2} A} \quad (\text{monotonicity})$$

The counit says: certified computations can be executed (via unbox). Monotonicity says: certification for r_1 implies certification for any $r_2 \succeq r_1$.

Cost-aware boxing. Promotion is conditional: $\text{box}_s(t) : \Box_s A$ requires $b \preceq s$ where b is the synthesized bound of t . There is no unconditional $A \rightarrow \Box_s A$.

4 Syntactic Term Model in $\mathbf{Set}^L$

We provide a semantic interpretation via a *syntactic term model* in the presheaf topos $\mathbf{Set}^L$. Types are presheaves of definable values indexed by resource bounds, the lattice is internalized, and cost extraction is a natural transformation.

4.1 The Presheaf Topos

Definition 4.1 (The topos $\mathbf{Set}^L$). *View the resource lattice $(L, \preceq)$ as a category:*

- *Objects: elements $r \in L$*
- *Morphisms: $r_1 \rightarrow r_2$ exists (uniquely) iff $r_1 \preceq r_2$*
- *Composition: transitivity of $\preceq$*

The presheaf topos $\mathbf{Set}^L$ has:

- *Objects: covariant functors $F : L \rightarrow \mathbf{Set}$*
- *Morphisms: natural transformations $\alpha : F \Rightarrow G$*

For a presheaf F , each $r \in L$ gives a set $F(r)$, and each $r_1 \preceq r_2$ gives a transition map $F(r_1 \preceq r_2) : F(r_1) \rightarrow F(r_2)$ satisfying functoriality.

We use covariant functors because resource bounds are cumulative: computations fitting budget r_1 also fit $r_2 \succeq r_1$. This monotonicity is captured by the transition maps.

4.2 The Internal Lattice

Definition 4.2 (The downset presheaf). *Define $\mathbb{L} : L \rightarrow \mathbf{Set}$ by:*

$$\mathbb{L}(r) = \downarrow r = \{a \in L \mid a \preceq r\}$$

For $r_1 \preceq r_2$, the transition map $\mathbb{L}(r_1 \preceq r_2) : \mathbb{L}(r_1) \rightarrow \mathbb{L}(r_2)$ is the inclusion $\iota_{r_1, r_2}(a) = a$, which is well-defined since $a \preceq r_1 \preceq r_2$ implies $a \preceq r_2$.

Lemma 4.3. *The lattice operations induce natural transformations:*

- $\tilde{\oplus} : \mathbb{L} \times \mathbb{L} \Rightarrow \mathbb{L}$ with $\tilde{\oplus}_r(a, b) = a \oplus b$
- $\tilde{\sqcup} : \mathbb{L} \times \mathbb{L} \Rightarrow \mathbb{L}$ with $\tilde{\sqcup}_r(a, b) = a \sqcup b$
- $\tilde{\perp} : \mathbf{1} \Rightarrow \mathbb{L}$ with $\tilde{\perp}_r(*) = \perp$

Proof. For $\tilde{\oplus}$: if $a, b \preceq r$ then $a \oplus b \preceq r$ by monotonicity of $\oplus$. Naturality follows since transition maps are inclusions and $\oplus$ is defined uniformly on L .

For $\tilde{\sqcup}$: if $a, b \preceq r$ then $a \sqcup b \preceq r$ since $a \sqcup b$ is the least upper bound and r is an upper bound of both.

For $\tilde{\perp}$: $\perp \preceq r$ for all r since $\perp$ is the least element. □

4.3 Type Interpretation

Definition 4.4 (Syntactic type interpretation). *For each type A , define $\llbracket A \rrbracket : L \rightarrow \mathbf{Set}$ by:*

$$\llbracket A \rrbracket(r) = \{(v, b) \in \mathbf{Val}_A \times L \mid b \preceq r, \emptyset \vdash_{r;b} v : A\}$$

For $r_1 \preceq r_2$, the transition map is inclusion: $(v, b) \mapsto (v, b)$.

Lemma 4.5. *$\llbracket A \rrbracket$ is a well-defined presheaf.*

Proof. The transition map is well-defined: if $(v, b) \in \llbracket A \rrbracket(r_1)$, then $b \preceq r_1 \preceq r_2$, and by budget weakening (Lemma 3.5), $\emptyset \vdash_{r_2;b} v : A$, so $(v, b) \in \llbracket A \rrbracket(r_2)$.

Functoriality is immediate: transition maps are inclusions, which compose associatively and have identity as the identity inclusion. $\square$

Elements (v, b) are pairs of a *definable value* v and its *certified bound* b . The bound b is the potential/worst-case cost from typing, not the operational cost (which is $\perp$ for values by rule (Val)).

Definition 4.6 (Cost natural transformation). *Define $\mathbf{cost} : \llbracket A \rrbracket \Rightarrow \mathbb{L}$ with components $\mathbf{cost}_r(v, b) = b$.*

Theorem 4.7. *$\mathbf{cost}$ is a natural transformation.*

Proof. We must show the naturality square commutes for $r_1 \preceq r_2$:

$$\begin{array}{ccc} \llbracket A \rrbracket(r_1) & \xrightarrow{\mathbf{cost}_{r_1}} & \mathbb{L}(r_1) \\ \downarrow & & \downarrow \\ \llbracket A \rrbracket(r_2) & \xrightarrow{\mathbf{cost}_{r_2}} & \mathbb{L}(r_2) \end{array}$$

For $(v, b) \in \llbracket A \rrbracket(r_1)$:

- Right-then-down: $\mathbf{cost}_{r_1}(v, b) = b$, then $\iota_{r_1, r_2}(b) = b$.
- Down-then-right: $\iota_{r_1, r_2}(v, b) = (v, b)$, then $\mathbf{cost}_{r_2}(v, b) = b$.

Both paths yield b , so the square commutes. $\square$

4.4 Type Constructors

Base types.

$$\begin{aligned} \llbracket \mathbf{Bool} \rrbracket(r) &= \{(\mathbf{tt}, \perp), (\mathbf{ff}, \perp)\} \\ \llbracket \mathbf{Nat} \rrbracket(r) &= \{(n, \perp) \mid n \in \mathbb{N}\} \end{aligned}$$

Transition maps are identities since $\perp \preceq r$ for all r .

Products.

$$\llbracket A \times B \rrbracket(r) = \left\{ ((v_1, v_2), b_1 \oplus b_2) \left| \begin{array}{l} (v_1, b_1) \in \llbracket A \rrbracket(r) \\ (v_2, b_2) \in \llbracket B \rrbracket(r) \\ b_1 \oplus b_2 \preceq r \end{array} \right. \right\}$$

This uses the categorical product structure and internal $\tilde{\oplus}$ to combine costs.

Functions (definable arrow space). Intuitively, functions in $\llbracket A \rightarrow B \rrbracket$ must map any definable argument to a definable result with appropriate cost bounds. This is the semantic counterpart of the (App) typing rule.

Definition 4.8. $\llbracket A \rightarrow B \rrbracket(r)$ consists of pairs $(\lambda x:A. t, b_{\text{body}})$ where:

1. $\emptyset \vdash_{r; b_{\text{body}}} \lambda x:A. t : A \rightarrow B$
2. For all $r_a \in L$ and $(v, b_a) \in \llbracket A \rrbracket(r_a)$, there exist $r_b \in L$ and $(w, b_b) \in \llbracket B \rrbracket(r_b)$ such that $t[x := v] \Downarrow w \triangleright k_b$ with $k_b \preceq b_b$ and $b_a \oplus b_b \oplus \delta_{\text{app}} \preceq r$.

Transition maps are inclusions: if the conditions hold at r_1 , they hold at $r_2 \succeq r_1$.

This is a sub-presheaf of cost-bounded definable functions, not necessarily the full exponential object in $\mathbf{Set}^L$.

Box modality.

$$\llbracket \Box_s A \rrbracket(r) = \{(\text{box}_s(v), b) \mid (v, b) \in \llbracket A \rrbracket(r), b \preceq s\}$$

Lemma 4.9. $\llbracket \Box_s A \rrbracket$ is a sub-presheaf of $\llbracket A \rrbracket$.

Proof. If $(\text{box}_s(v), b) \in \llbracket \Box_s A \rrbracket(r_1)$ with $b \preceq s$, then $(v, b) \in \llbracket A \rrbracket(r_2)$ for $r_2 \succeq r_1$ by monotonicity of $\llbracket A \rrbracket$. The constraint $b \preceq s$ is independent of budget, so $(\text{box}_s(v), b) \in \llbracket \Box_s A \rrbracket(r_2)$. $\square$

The constraint $b \preceq s$ enforces certification: you cannot box a value whose bound exceeds the claimed grade.

4.5 Semantic Soundness

Theorem 4.10 (Semantic soundness). *If $\emptyset \vdash_{r; b} t : A$ and $t \Downarrow v \triangleright k$, then:*

1. $k \preceq b$ (by Theorem 3.9)
2. $(v, b) \in \llbracket A \rrbracket(b)$
3. $\text{cost}((v, b)) = b$

Proof. By induction on the typing derivation, using Theorem 3.9 for part (1).

Base case (Lam). $t = \lambda x.s$ is a value with $x:A \vdash_{r; b} s : B$. The value $\lambda x.s$ has certified bound b (latent cost of the body). By Definition 4.8, we must verify the application condition: for any $(v, b_a) \in \llbracket A \rrbracket(r_a)$, by Lemma 3.4, $s[x := v] \Downarrow w \triangleright k_b$ with $k_b \preceq b$. Thus $(\lambda x.s, b) \in \llbracket A \rightarrow B \rrbracket(b)$.

Inductive case (App). By IH, $f \Downarrow \lambda x.s \triangleright k_f$ with $(\lambda x.s, b_f) \in \llbracket A \rightarrow B \rrbracket(b_f)$, and $a \Downarrow v \triangleright k_a$ with $(v, b_a) \in \llbracket A \rrbracket(b_a)$.

By Definition 4.8, applying $(\lambda x.s, b_f)$ to (v, b_a) produces $(w, b_b) \in \llbracket B \rrbracket(b_b)$ with $s[x := v] \Downarrow w \triangleright k_b$ and $k_b \preceq b_b$.

The synthesized bound is $b = b_f \oplus b_a \oplus \delta_{\text{app}}$. Since $(w, b_b) \in \llbracket B \rrbracket(b_b)$ and $b_b \preceq b$ (by the cost accounting), we have $(w, b) \in \llbracket B \rrbracket(b)$ by the transition map.

Inductive case (Pair). By IH, $t_1 \Downarrow v_1 \triangleright k_1$ with $(v_1, b_1) \in \llbracket A \rrbracket(b_1)$, and $t_2 \Downarrow v_2 \triangleright k_2$ with $(v_2, b_2) \in \llbracket B \rrbracket(b_2)$.

By the product interpretation, $(\langle v_1, v_2 \rangle, b_1 \oplus b_2) \in \llbracket A \times B \rrbracket(b_1 \oplus b_2)$.

Since $b = b_1 \oplus b_2$, we have $(\langle v_1, v_2 \rangle, b) \in \llbracket A \times B \rrbracket(b)$.

Inductive case (Box). By IH, $t \Downarrow v \triangleright k$ with $(v, b) \in \llbracket A \rrbracket(b)$ and $k \preceq b$.

The (Box) rule requires $b \preceq s$. By the box interpretation, $(\text{box}_s(v), b) \in \llbracket \Box_s A \rrbracket(b)$ since $b \preceq s$.

Other cases. Similar reasoning using IH and the type constructor definitions.

Part (3) is immediate: $\text{cost}_b(v, b) = b$ by Definition 4.6. $\square$

4.6 Canonical Forms via Reification

Definition 4.11 (Reification). *The reification function $\text{reify}_A : \llbracket A \rrbracket(r) \rightarrow \text{Val}_A$ extracts the value component: $\text{reify}_A(v, b) = v$.*

Theorem 4.12 (Canonical forms). *For all types A , bounds $b \in L$, and elements $(v, b) \in \llbracket A \rrbracket(r)$:*

1. $\emptyset \vdash_{r; b} \text{reify}_A(v, b) : A$
2. $\text{reify}_A(v, b) = v$ (values are their own reifications)
3. $v \Downarrow v \triangleright \perp$ (values evaluate with cost $\perp$)

Proof. By induction on the structure of type A .

Base case ($A = \text{Bool}$). $\llbracket \text{Bool} \rrbracket(r) = \{(\text{tt}, \perp), (\text{ff}, \perp)\}$, so $v \in \{\text{tt}, \text{ff}\}$ and $b = \perp$.

- (1) By definition of $\llbracket \text{Bool} \rrbracket$, $\emptyset \vdash_{r; \perp} v : \text{Bool}$.
- (2) $\text{reify}_{\text{Bool}}(v, \perp) = v$ by definition.
- (3) By rule (Val), $v \Downarrow v \triangleright \perp$.

Inductive case ($A = A_1 \times A_2$). By definition: $(v_1, b_1) \in \llbracket A_1 \rrbracket(r)$, $(v_2, b_2) \in \llbracket A_2 \rrbracket(r)$, $v = \langle v_1, v_2 \rangle$, $b = b_1 \oplus b_2$.

- By IH: $\emptyset \vdash_{r; b_i} v_i : A_i$ and $v_i \Downarrow v_i \triangleright \perp$.
- (1) By (Pair): $\emptyset \vdash_{r; b_1 \oplus b_2} \langle v_1, v_2 \rangle : A_1 \times A_2$.
 - (2) $\text{reify}_{A_1 \times A_2}(\langle v_1, v_2 \rangle, b) = \langle v_1, v_2 \rangle = v$.
 - (3) By rule (Val): $\langle v_1, v_2 \rangle \Downarrow \langle v_1, v_2 \rangle \triangleright \perp$.

Inductive case ($A = A_1 \rightarrow A_2$). $v = \lambda x. t$ with $(\lambda x. t, b) \in \llbracket A_1 \rightarrow A_2 \rrbracket(r)$.

- (1) By Definition 4.8, $\emptyset \vdash_{r; b} \lambda x. t : A_1 \rightarrow A_2$.
- (2) $\text{reify}_{A_1 \rightarrow A_2}(\lambda x. t, b) = \lambda x. t = v$.
- (3) By rule (Val): $\lambda x. t \Downarrow \lambda x. t \triangleright \perp$.

Inductive case ($A = \Box_s A'$). $v = \text{box}_s(v')$ with $(v', b) \in \llbracket A' \rrbracket(r)$ and $b \preceq s$.

- By IH: $\emptyset \vdash_{r; b} v' : A'$ and $v' \Downarrow v' \triangleright \perp$.
- (1) By (Box) with $b \preceq s$: $\emptyset \vdash_{r; b} \text{box}_s(v') : \Box_s A'$.
 - (2) $\text{reify}_{\Box_s A'}(\text{box}_s(v'), b) = \text{box}_s(v') = v$.
 - (3) By rule (Box): $\text{box}_s(v') \Downarrow \text{box}_s(v') \triangleright \perp$ (since $v' \Downarrow v' \triangleright \perp$). $\square$

Corollary 4.13 (Adequacy). *The syntactic term model in Set^L is adequate:*

- **Soundness:** $\emptyset \vdash_{r; b} t : A$ and $t \Downarrow v \triangleright k$ implies $(v, b) \in \llbracket A \rrbracket(b)$ (Theorem 4.10)
- **Canonical forms:** $(v, b) \in \llbracket A \rrbracket(r)$ implies v is the canonical representation with $\emptyset \vdash_{r; b} v : A$ (Theorem 4.12)

The correspondence is exact: every definable value in the presheaf model has a unique syntactic representative (itself), and every typed value inhabits the presheaf model.

4.7 Universal Property: Initiality

We establish that the syntactic model is *initial* in the category of resource-bounded models: it embeds uniquely into all such models.

Definition 4.14 (Resource-bounded model). *A resource-bounded model $\mathcal{M}$ consists of:*

1. *A category $\mathcal{C}$ with finite products*
2. *An internal lattice $L_{\mathcal{M}} \in \mathcal{C}$ with morphisms $\oplus_{\mathcal{M}}, \sqcup_{\mathcal{M}} : L_{\mathcal{M}} \times L_{\mathcal{M}} \rightarrow L_{\mathcal{M}}$ and $\perp_{\mathcal{M}} : 1 \rightarrow L_{\mathcal{M}}$*
3. *For each type A , an object $\llbracket A \rrbracket_{\mathcal{M}} \in \mathcal{C}$*
4. *For each type A , a cost morphism $\text{cost}_{\mathcal{M}} : \llbracket A \rrbracket_{\mathcal{M}} \rightarrow L_{\mathcal{M}}$*
5. *Definable CCC structure: eval and curry morphisms on the chosen type-objects (not necessarily the ambient exponentials)*
6. $\square_r$ *subobjects satisfying counit ($\llbracket \square_r A \rrbracket_{\mathcal{M}} \rightarrow \llbracket A \rrbracket_{\mathcal{M}}$) and monotonicity ($r_1 \preceq r_2$ implies $\llbracket \square_{r_1} A \rrbracket_{\mathcal{M}} \rightarrow \llbracket \square_{r_2} A \rrbracket_{\mathcal{M}}$)*

The key point: we require CCC structure on the *chosen* type-objects, not that $\llbracket A \rightarrow B \rrbracket_{\mathcal{M}}$ be the full exponential in $\mathcal{C}$. This allows models with definable, cost-bounded functions.

Example 4.15. *The syntactic model $\mathbf{Set}^L$ is a resource-bounded model with $\mathcal{C} = \mathbf{Set}^L$, using the constructions of §4.*

Example 4.16. *Any set-theoretic interpretation where types are sets with cost functions, products are cartesian, functions are cost-bounded maps with curry/eval, and box is defined via cost constraints forms a resource-bounded model with $\mathcal{C} = \mathbf{Set}$.*

Construction 4.17 (Interpretation functor). *Given a resource-bounded model $\mathcal{M}$, define $\llbracket - \rrbracket_{\mathcal{M}}$ by mutual recursion:*

Types. *Given by Definition 4.14.*

Contexts. $\llbracket x_1:A_1, \dots, x_n:A_n \rrbracket_{\mathcal{M}} = \llbracket A_1 \rrbracket_{\mathcal{M}} \times \dots \times \llbracket A_n \rrbracket_{\mathcal{M}}$

Terms. *For $\Gamma \vdash_{r,b} t : A$, define $\llbracket t \rrbracket_{\mathcal{M}} : \llbracket \Gamma \rrbracket_{\mathcal{M}} \rightarrow \llbracket A \rrbracket_{\mathcal{M}}$ by induction:*

- $\llbracket x_i \rrbracket_{\mathcal{M}} = \pi_i$ (projection)
- $\llbracket \lambda x. t \rrbracket_{\mathcal{M}} = \text{curry}(\llbracket t \rrbracket_{\mathcal{M}})$
- $\llbracket f a \rrbracket_{\mathcal{M}} = \text{eval} \circ \langle \llbracket f \rrbracket_{\mathcal{M}}, \llbracket a \rrbracket_{\mathcal{M}} \rangle$
- $\llbracket (s, t) \rrbracket_{\mathcal{M}} = \langle \llbracket s \rrbracket_{\mathcal{M}}, \llbracket t \rrbracket_{\mathcal{M}} \rangle$
- $\llbracket \pi_i t \rrbracket_{\mathcal{M}} = \pi_i \circ \llbracket t \rrbracket_{\mathcal{M}}$
- $\llbracket \text{box}_r(t) \rrbracket_{\mathcal{M}} = \text{inclusion into } \llbracket \square_r A \rrbracket_{\mathcal{M}} \text{ (using } b \preceq r \text{)}$
- $\llbracket \text{unbox}(t) \rrbracket_{\mathcal{M}} = \text{counit applied to } \llbracket t \rrbracket_{\mathcal{M}}$

Lemma 4.18 (Well-definedness). *The interpretation $\llbracket - \rrbracket_{\mathcal{M}}$ is well-defined: if two derivations prove the same judgment, their interpretations are equal.*

Proof. By induction on typing derivations. The interpretation is determined by the categorical structure (products, CCC, subobjects) in Definition 4.14, which gives unique morphisms. $\square$

Theorem 4.19 (Type preservation). *For any $\Gamma \vdash_{r;b} t : A$, $\llbracket t \rrbracket_{\mathcal{M}} : \llbracket \Gamma \rrbracket_{\mathcal{M}} \rightarrow \llbracket A \rrbracket_{\mathcal{M}}$ is a valid morphism.*

Proof. By construction of $\llbracket - \rrbracket_{\mathcal{M}}$: each clause produces a morphism of the correct type using the categorical structure. $\square$

Theorem 4.20 (Cost preservation). *For closed $\emptyset \vdash_{r;b} t : A$ with $t \Downarrow v \triangleright k$ and $k \preceq b$:*

$$\text{cost}_{\mathcal{M}}(\llbracket t \rrbracket_{\mathcal{M}}) \preceq \llbracket b \rrbracket_{\mathcal{M}}$$

in the internal order of $L_{\mathcal{M}}$.

Proof. By induction on the typing derivation, using:

- Cost soundness (Theorem 3.9): $k \preceq b$
- The interpretation composes costs according to typing rules
- Internal lattice operations in $\mathcal{M}$ satisfy the same axioms as L

For (App): $\text{cost}(\llbracket f a \rrbracket_{\mathcal{M}}) \preceq \text{cost}(\llbracket f \rrbracket_{\mathcal{M}}) \oplus_{\mathcal{M}} \text{cost}(\llbracket a \rrbracket_{\mathcal{M}}) \oplus_{\mathcal{M}} \delta_{\text{app}}$, matching the typing rule.

For (Box): by the subobject definition, $\llbracket \text{box}_s(t) \rrbracket_{\mathcal{M}}$ has cost $\preceq s$. $\square$

Theorem 4.21 (Uniqueness). *Any two interpretations $\llbracket - \rrbracket_{\mathcal{M}}^{(1)}$ and $\llbracket - \rrbracket_{\mathcal{M}}^{(2)}$ agreeing on base types agree on all types and terms.*

Proof. By induction on structure:

Types.

- Base types: by assumption
- Products: forced by categorical product (unique up to isomorphism)
- Functions: forced by CCC structure (curry/eval are unique)
- Box: forced by subobject definition

Terms. Each typing rule corresponds to a unique categorical operation:

- Variables $\mapsto$ projections (unique)
- Lambda $\mapsto$ curry (unique by CCC universal property)
- Application $\mapsto$ eval (unique by CCC universal property)
- Pairing $\mapsto$ product introduction (unique by product universal property)
- Box/Unbox $\mapsto$ subobject inclusion/counit (unique)

Therefore any two interpretations must coincide. $\square$

Theorem 4.22 (Initiality). *The syntactic model $\mathbf{Syn} = \mathbf{Set}^L$ is initial in the category of resource-bounded models: for any model $\mathcal{M}$, there exists a unique structure-preserving morphism $\llbracket - \rrbracket_{\mathcal{M}} : \mathbf{Syn} \rightarrow \mathcal{M}$.*

Proof. • **Existence:** Construction 4.17

- **Well-definedness:** Lemma 4.18
- **Preservation:** Theorems 4.19 and 4.20
- **Uniqueness:** Theorem 4.21

□

Corollary 4.23 (Embedding into arbitrary models). *For any closed $\emptyset \vdash_{r;b} t : A$ and any model $\mathcal{M}$:*

$$\llbracket t \rrbracket_{\mathcal{M}} \in \llbracket A \rrbracket_{\mathcal{M}} \quad \text{with} \quad \text{cost}_{\mathcal{M}}(\llbracket t \rrbracket_{\mathcal{M}}) \preceq b$$

Every typed term has a canonical interpretation in every model with costs preserved. The syntactic model is the free resource-bounded model.

4.8 Concrete Instantiations

Time only. $L = \mathbb{N}$, $\oplus = +$, $\sqcup = \max$. Theorem 3.9 becomes $k \leq b \leq r$.

Multi-dimensional. $L = \mathbb{N}^3$ (time, memory, depth) with pointwise operations bounds all dimensions simultaneously.

Gas. $L = \mathbb{N}$ interpreted as gas units; only δ constants change per EVM version.

5 Recursion Patterns

The metatheory covers recursion-free terms. For recursive functions, we use Lean’s well-founded recursion with separate bound proofs.

Listing 1: Binary search with termination proof

```
def binarySearch (arr : Array Nat) (target : Nat) : Option Nat :=
  if h : arr.isEmpty then none
  else
    let mid := arr.size / 2
    let pivot := arr[mid]
    match target.compare pivot with
    | .eq => some mid
    | .lt => binarySearch (arr.slice 0 mid) target
    | .gt => binarySearch (arr.slice (mid + 1) arr.size) target
  termination_by arr.size
```

Bounds are proven separately and composed:

```
theorem binarySearch_bound (arr : Array a) (target : a) :
  cost (binarySearch arr target) <= log arr.size + 5 := by
  -- well-founded induction on arr.size
```

This approach does not automatically synthesize size-indexed bounds; full dependent types are future work.

6 Engineering in Lean 4

We discuss two integration approaches. Code is available at <https://github.com/CoreyThuro/RB-TT>.

Deep embedding. Formalize the calculus inside Lean: define syntax, typing judgments, and evaluation as inductive types/relations; prove Theorems 3.9–4.22 mechanically. This validates correctness but is ongoing work.

Shallow embedding (current). Write Lean programs with custom `@[budget_bound]` annotations; prove bounds as separate theorems; use CI to check structural cost proxies.

The correspondence gap. The formal bound b from typing rules is abstract; the practical cost model $\text{cost}(t) = \alpha \cdot \text{nodes}(t) + \beta \cdot \text{lamDepth}(t) + \dots$ measures elaborated terms. These are separate models. The CI system uses practical costs for regression testing without claiming $\text{cost}(\text{elab}(t)) \leq b$. Despite this gap, the combination is useful: developers reason compositionally, CI catches regressions, and empirical data can refine heuristics.

7 Case Study: Binary Search

We instantiate with $L = \mathbb{N}$. Binary search has bound $b = \lceil \log_2 n \rceil + 5$.

Semantic validation. By Theorem 4.10, $(\text{binarySearch}, b) \in \llbracket \text{Array Nat} \rightarrow \text{Option Nat} \rrbracket(b)$. By Theorem 4.12, values are their own reifications. By Theorem 4.22, the interpretation embeds into all models.

Portability. The same proof structure works for multi-dimensional lattices (time, memory, depth) or gas budgets by changing δ constants.

Lessons. Compositional bounds via recursion patterns work; cost soundness is certified; the syntactic model validates composition; practical proxies catch regressions. Caveats: bounds are proven separately; the CI model is empirical; automatic synthesis is future work.

8 Limitations and Future Work

What we provide. Compositional framework for recursion-free STLC; syntactic model in Set^L ; cost soundness, canonical forms, and initiality theorems; recursion via separate proofs; structural cost proxies.

Open problems. Dependent types with size-indexed bounds; formal correspondence to elaboration; probabilistic/average-case bounds; full mechanization; HoTT extensions.

Future directions. Complete mechanization; extend to dependent types; cost-aware elaborator; tooling for Coq/Agda; empirical validation; graded parametricity; integration with RAML.

9 Related Work

Type theory foundations. Our work builds on the tradition of Martin-Löf type theory [1, 2] and the Curry-Howard correspondence between proofs and programs [3]. Pierce [4] provides a comprehensive treatment of type systems for programming languages, which informs our typing rules.

Categorical semantics and topoi. The categorical approach to type theory originates with Lambek and Scott [5]. Our use of presheaf topoi follows Mac Lane and Moerdijk [6]; Johnstone [7] provides the definitive reference for topos theory. Jacobs [8] develops the connection between categorical logic and type theory systematically. Our syntactic model in $\mathbf{Set}^L$ uses standard presheaf constructions with the resource lattice as the indexing category.

Coeffects and graded modalities. Petricek et al. [9] introduced coeffects for context-dependent computation; Brunel et al. [10] developed graded comonads; Orchard et al. [11] apply graded types in Granule. Gaboardi et al. [12] combine effects and coeffects via grading. We generalize to abstract lattices, distinguish $\oplus$ from $\sqcup$, and provide a syntactic model with initiality.

Linear logic and bounded complexity. Girard’s linear logic [15] introduced resource-sensitive reasoning. Benton [16] developed mixed linear/non-linear systems. Hofmann [13] and Dal Lago–Hofmann [14] use linear types for complexity bounds. We target explicit budgets rather than complexity classes.

Quantitative type theory. Atkey [17] tracks usage multiplicities; McBride [18] explores quantity-aware programming. We track computational cost via presheaves and natural transformations rather than usage counts.

Automatic resource analysis. Hoffmann et al. [19, 20] infer polynomial bounds via RAML using potential functions. We certify bounds via typing with categorical validation rather than inference.

Separation and indexed models. O’Hearn et al. [22] and Reynolds [23] reason about heap resources. Appel–McAllester [21] use step-indexing for recursive types. Our presheaves track certified bounds with initiality rather than step indices.

Proof assistants. Our engineering integration targets Lean 4 [24]. Coq [25] and other proof assistants provide alternative platforms for mechanization.

10 Conclusion

We presented a compositional framework for resource-bounded types using abstract lattices, formalized for recursion-free STLC. Key contributions: (1) cost soundness (Theorem 3.9) by structural induction; (2) a syntactic model in $\mathbf{Set}^L$ with cost as a natural transformation (Theorem 4.10); (3) canonical forms (Theorem 4.12); (4) initiality establishing universal completeness (Theorem 4.22).

The initiality result shows our typing rules correctly capture resource-bounded computation: the syntactic model is the free resource-bounded model. Engineering integration uses structural proxies, acknowledging the gap with elaborated terms. The framework provides a rigorous foundation for reasoning about resource bounds compositionally, with applications in safety-critical systems, blockchain, and embedded devices.

Future work includes mechanization, dependent types with automatic bound synthesis, and integration with existing tools like RAML.

References

- [1] P. Martin-Löf. *Intuitionistic Type Theory*. Bibliopolis, Naples, 1984. Notes by Giovanni Sambin of a series of lectures given in Padua, June 1980.

- [2] B. Nordström, K. Petersson, and J. M. Smith. *Programming in Martin-Löf's Type Theory: An Introduction*. Oxford University Press, 1990.
- [3] J.-Y. Girard, Y. Lafont, and P. Taylor. *Proofs and Types*. Cambridge Tracts in Theoretical Computer Science. Cambridge University Press, 1989.
- [4] B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [5] J. Lambek and P. J. Scott. *Introduction to Higher-Order Categorical Logic*. Cambridge Studies in Advanced Mathematics. Cambridge University Press, 1986.
- [6] S. Mac Lane and I. Moerdijk. *Sheaves in Geometry and Logic: A First Introduction to Topos Theory*. Universitext. Springer-Verlag, 1992.
- [7] P. T. Johnstone. *Sketches of an Elephant: A Topos Theory Compendium*, volumes 1–2. Oxford Logic Guides. Oxford University Press, 2002.
- [8] B. Jacobs. *Categorical Logic and Type Theory*. Studies in Logic and the Foundations of Mathematics. Elsevier, 1999.
- [9] T. Petricek, D. Orchard, and A. Mycroft. Coeffects: A calculus of context-dependent computation. In *Proc. ICFP*, pages 123–135. ACM, 2014.
- [10] A. Brunel, M. Gaboardi, D. Mazza, and S. Zdancewic. A core quantitative coeffect calculus. In *Proc. ESOP*, volume 8410 of *LNCS*, pages 351–370. Springer, 2014.
- [11] D. Orchard, V.-B. Liepelt, and H. Eades III. Quantitative program reasoning with graded modal types. *Proc. ACM Program. Lang.*, 3(ICFP):110:1–110:30, 2019.
- [12] M. Gaboardi, S. Katsumata, D. Orchard, F. Breuvar, and T. Uustalu. Combining effects and coeffects via grading. In *Proc. ICFP*, pages 476–489. ACM, 2016.
- [13] M. Hofmann. Linear types and non-size-increasing polynomial time computation. *Inform. and Comput.*, 183(1):57–85, 2003.
- [14] U. Dal Lago and M. Hofmann. Bounded linear logic, revisited. *Log. Methods Comput. Sci.*, 6(4):1–31, 2010.
- [15] J.-Y. Girard. Linear logic. *Theoret. Comput. Sci.*, 50(1):1–102, 1987.
- [16] P. N. Benton. A mixed linear and non-linear logic: Proofs, terms and models. In *Proc. CSL*, volume 933 of *LNCS*, pages 121–135. Springer, 1995.
- [17] R. Atkey. Syntax and semantics of quantitative type theory. In *Proc. LICS*, pages 56–65. ACM, 2018.
- [18] C. McBride. I got plenty o' nuttin'. In *A List of Successes That Can Change the World*, volume 9600 of *LNCS*, pages 207–233. Springer, 2016.
- [19] J. Hoffmann, K. Aehlig, and M. Hofmann. Multivariate amortized resource analysis. *ACM Trans. Program. Lang. Syst.*, 39(1):3:1–3:58, 2017.
- [20] J. Hoffmann and M. Hofmann. Amortized resource analysis with polynomial potential. In *Proc. ESOP*, volume 6012 of *LNCS*, pages 287–306. Springer, 2010.
- [21] A. W. Appel and D. McAllester. An indexed model of recursive types for foundational proof-carrying code. *ACM Trans. Program. Lang. Syst.*, 23(5):657–683, 2001.

- [22] P. W. O’Hearn, J. C. Reynolds, and H. Yang. Local reasoning about programs that alter data structures. In *Proc. CSL*, volume 2142 of *LNCS*, pages 1–19. Springer, 2001.
- [23] J. C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Proc. LICS*, pages 55–74. IEEE, 2002.
- [24] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Proc. CADE*, volume 12699 of *LNCS*, pages 625–635. Springer, 2021.
- [25] The Coq Development Team. *The Coq Proof Assistant Reference Manual*, version 8.17, 2023. <https://coq.inria.fr>.